

0.113. #include <decaler> — a << b et a >> b

#include <decaler> : décaler les bits d'un nombre calculé de rangs — chaque rang à gauche double, chaque rang à droite divise par deux.

1. Ce qu'il faut comprendre

Ce que c'est : `a << b` décale les bits de `a` de `b` rangs vers la gauche : chaque rang **double** (1 << 3 vaut 8). `a >> b` décale vers la droite : chaque rang **divise par deux** (200 >> 2 vaut 50). Quand `b` est écrit en clair, le compilateur le fait seul ; quand `b` se calcule, il faut une routine.

La ligne à écrire : `#include <decaler>`.

Ce qu'elle coûte : une vingtaine d'octets par sens.

Essaie : `1 << n` pour n de 0 à 7 : 1, 2, 4, 8... 128.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **435 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      59 o ← la console, à l'allumage – la mise en route de la console
main()         49 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireNombre – attend de pouvoir écrire une case
EcrireNombre   33 o ← nombre() – écrit un nombre en chiffres
DecalerGauche  12 o ← main() – décale vers la gauche d'un nombre calculé de rangs
DecalerDroite  13 o ← main() – décale vers la droite d'un nombre calculé de rangs
CopierTuiles   31 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles        160 o ← CopierTuiles – 10 lettres sur 43
```

Lettres gravées : 0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, Diviser, Reste, Multiplier, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
nombre() × 2    20 o sur place, dans main() + EcrireNombre (33 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <decaler> : a << b et a >> b ----

#include <decaler> // les routines qui décalent d'un nombre calculé de rangs
#include <nombre>
```

2. UNE VARIABLE — N

```
uint8_t n = 3;
```

3. UNE VARIABLE — GAUCHE

```
uint8_t gauche = 0;
```

4. UNE VARIABLE — DROITE

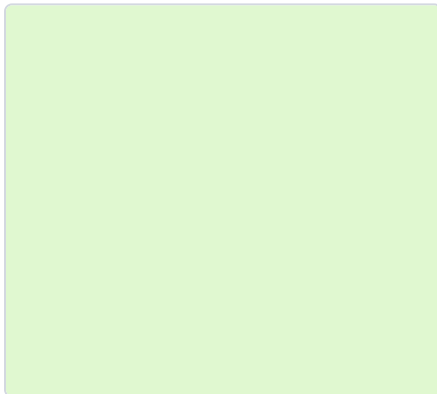
```
uint8_t droite = 0;
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    gauche = 1 << n;           // 1 décalé de 3 rangs : 8  
    droite = 200 >> (n - 1);    // 200 décalé de 2 rangs : 50  
    nombre(8, 6, gauche);       // 008  
    nombre(8, 8, droite);       // 050  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

008, puis 050 en dessous.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ `1 << 3 = 8`

✓ `200 >> 2 = 50`

✓ les deux s'affichent

